

Outbox, mensageria, webhooks e Pix

Guia de estudo para a entrevista. Baseado no projeto **nestjs-outbox** (NestJS + PostgreSQL + Apache Pulsar). Leitura em ~25 minutos.

1. Pitch de 1 minuto

2. O fluxo em 10 passos

3. Transactional Outbox

4. Idempotência

5. Relay
6. Pulsar e mensageria

7. Webhooks

8. Pix: SPI, pacs.008, pacs.002, pacs.004

9. Perguntas prováveis

10. Glossário e checklist da demo

1 Pitch de 1 minuto

"Construí uma API de pedidos que garante que **nenhum evento se perde e nenhum efeito é aplicado duas vezes**, mesmo com o broker fora do ar, processo caindo no meio ou mensagens duplicadas.

O pedido e o evento são gravados na **mesma transação** (Transactional Outbox). Um **relay** publica os eventos no **Pulsar** com retry e backoff. Os consumidores são **idempotentes** (inbox). O fluxo segue até uma **API externa de transportadora**, chamada com Idempotency-Key, que devolve um **webhook assinado** confirmando a entrega; no fim, o cliente recebe um **webhook** nosso, também assinado e com retry.

É exatamente o tipo de garantia que um arranjo como o **Pix** exige: a ordem de pagamento (pacs.008) não pode sair duas vezes nem se perder, e a confirmação (pacs.002) pode chegar repetida."

2 O fluxo em 10 passos

Ideia central: **cada passo grava o próximo passo na mesma transação em que termina**, e toda chamada que pode ser repetida é idempotente.

#	Passo	Quem	Garantia que mostra
1	Cliente chama <code>POST /orders</code> com <code>Idempotency-Key</code>	Cliente → API	Repetir a chamada não cria outro pedido (409 na duplicata)
2	Uma transação: pedido (PENDING) + evento <code>OrderCreated</code> na outbox	API → Postgres	Atomicidade: não existe pedido sem evento
3	Relay publica <code>OrderCreated</code> no Pulsar	Relay → Pulsar	Retry com backoff se o broker cair
4	Consumidor confirma o pedido (CONFIRMED = PAID)	Pulsar → Consumidor	Inbox: duplicata do broker é descartada
5	Mesma transação grava <code>OrderConfirmed</code> na outbox + webhook <code>order.confirmed</code>	Consumidor → Postgres	Outbox de novo, um salto adiante
6	Relay publica <code>OrderConfirmed</code>	Relay → Pulsar	Encadeamento de eventos
7	Serviço de envio chama a API da transportadora → 202, pedido SHIPPED	Envio → API externa	Idempotency-Key = eventId; falha = NACK e nova tentativa
8	Transportadora faz a entrega	Transportadora	Processo assíncrono: não esperamos bloqueados

9	Webhook <code>shipment.delivered</code> chega assinado; pedido DELIVERED	Transportadora → API	HMAC + id único: reenvio não reprocessa
10	Cliente final recebe webhook <code>order.delivered</code>	API → Cliente	Webhook com assinatura, retry e histórico

3 Transactional Outbox

O problema: dual write

Gravar no banco e publicar no broker são duas operações em sistemas diferentes. Não existe transação que abranja os dois:

- Grava no banco e cai antes de publicar → **evento perdido** (pedido existe, ninguém fica sabendo).
- Publica e o commit falha → **evento fantasma** (avisou sobre um pedido que não existe).

A solução

```
BEGIN;  
  INSERT INTO orders (...) VALUES (...);           -- mudança de estado  
  INSERT INTO outbox (event_type, payload, ...)      -- evento, mesma transação  
    VALUES ('OrderCreated', '{...}');  
COMMIT;                                           -- ou os dois, ou nenhum  
-- Depois, um processo separado (relay) lê a outbox e publica no broker.
```

Frase para a entrevista

"A outbox transforma um problema de consistência distribuída num problema local: o banco garante a atomicidade, e o relay garante a entrega com at-least-once."

Pontos de atenção

- **Entrega é at-least-once**, não exactly-once: o relay pode publicar e cair antes de marcar. Por isso o consumidor precisa ser idempotente.
- **Ordem**: eventos do mesmo agregado usam a mesma chave (`aggregate_id`) para manter a ordem no broker.
- **Crescimento da tabela**: expurgo diário dos eventos publicados (`OUTBOX_RETENTION_DAYS`).
- **Alternativa**: CDC (Debezium lendo o WAL do Postgres). Menos latência, mais infraestrutura.

4 Idempotência

Definição: executar a mesma operação N vezes tem o mesmo efeito que executar uma vez. Em sistemas distribuídos, retry é inevitável (timeout não diz se a operação aconteceu), então idempotência é obrigatória.

Onde	Como foi feito	Chave
API (entrada)	Tabela <code>idempotency_keys</code> gravada na mesma transação do pedido. Mesma chave + mesmo corpo → 409 "transação duplicada"; corpo diferente → 422. A chave primária serializa requisições simultâneas.	<code>Idempotency-Key</code> do cliente
Consumidor (inbox)	<code>INSERT INTO processed_events ... ON CONFLICT DO NOTHING</code> na mesma transação do efeito. Se não inseriu, é duplicata: pula o efeito e dá ACK.	(<code>event_id</code> , <code>consumer</code>)
Chamada a API externa	Envia <code>Idempotency-Key</code> = id do evento. Se o consumidor reprocessar, a transportadora devolve a mesma remessa.	<code>eventId</code>
Webhook que enviamos	Header <code>webhook-id</code> igual em todos os retries: o cliente deduplica por ele.	id da entrega
Webhook que recebemos	Índice único (<code>source</code> , <code>external_event_id</code>) em <code>inbound_webhooks</code> ; reenvio → 200 sem reprocessar.	id do evento do parceiro

Detalhe que impressiona

Responder a duplicata com **2xx** (e não erro) no webhook recebido: se respondermos erro, o parceiro vai continuar reenviando para sempre.

Quando a transação vira PAID

A API expõe `paymentStatus` (`PENDING` ou `PAID`) e `paidAt` . A regra é uma só: **PAID quando o consumidor confirmou o pedido** (`confirmedAt` preenchido). Como a confirmação é assíncrona, o `POST` **nunca** responde PAID:

Momento	Onde o cliente vê	JSON
Na hora do pedido	Resposta do <code>POST /orders</code>	<code>"status": "PENDING", "paymentStatus": "PENDING", "paidAt": null</code>
Depois do consumidor (~1s)	Consulta: <code>GET /orders/:id</code>	<code>"paymentStatus": "PAID", "paidAt": "2026-10-06T17:17:49.809Z"</code>
Depois do consumidor (~1s)	Webhook <code>order.confirmed</code> (push, assinado)	<code>"status": "CONFIRMED", "paymentStatus": "PAID", "paidAt": "..."</code>

Frase para a entrevista

"O POST responde PENDING porque o pagamento é confirmado de forma assíncrona. O PAID chega por webhook ou por consulta, como a pacs.002 com ACSC no Pix. Fazer a API esperar a confirmação jogaria fora a vantagem do outbox: responder rápido mesmo com o broker fora do ar."

PAID e entrega são independentes: um pedido `SHIPPED` ou `DELIVERED` continua `PAID` .

5 Relay (message relay)

Processo que lê a outbox a cada 1s e publica no broker, em três passos:

```
1. CLAIM  UPDATE outbox SET status='processing', attempts=attempts+1,
          locked_until = now() + lease
          WHERE id IN (SELECT id FROM outbox
                      WHERE (status='pending' AND available_at <= now())
                          OR (status='processing' AND locked_until < now())
                      ORDER BY created_at LIMIT 50
                      FOR UPDATE SKIP LOCKED)
2. PUBLISH fora da transação (key = aggregate_id)
3. MARK    published | pending + backoff | failed (dead letter)
```

- **FOR UPDATE SKIP LOCKED:** vários relays rodam em paralelo sem pegar a mesma linha (escala horizontal sem líder).
- **Lease (locked_until):** se o relay morrer entre publicar e marcar, o lease expira e outro relay reprocessa. Gera duplicata, que o consumidor descarta.
- **Backoff exponencial com jitter:** 1s, 2s, 4s, 8s... + até 1s aleatório. O jitter evita que eventos que falharam juntos sejam retentados juntos (thundering herd).
- **Dead letter:** depois de `OUTBOX_MAX_ATTEMPTS`, status `failed`; volta para a fila manualmente.
- **Transações curtas:** nunca segurar lock no banco enquanto espera a rede.

6 Pulsar e mensageria

Como o Pulsar recebe e entrega uma mensagem

1. **Producer** (o relay) envia ao **broker** dono do tópico `persistent://public/default/order.created`.
2. O broker grava a mensagem como uma **entry** num **ledger** do **BookKeeper** (camada de armazenamento separada).
3. Só depois de gravada o broker devolve o **send receipt** com o `messageId` (ledger, entry). Aí o relay marca `published`.
4. A **assinatura** (subscription) guarda um **cursor** e entrega aos consumidores.
5. **ACK** avança o cursor e tira a mensagem do backlog. **NACK** faz o broker reentregar (aqui em ~1s).
6. Depois de 10 reentregas, a mensagem vai para o tópico de **DLQ** (`order.created.order-confirmation.DLQ`).

Tipos de assinatura

Tipo	Comportamento
Exclusive	Um único consumidor. Ordem total.
Failover	Um ativo, outros em espera.
Shared	Round-robin entre consumidores. Sem garantia de ordem.
Key_Shared (usado)	Mesma chave → sempre o mesmo consumidor. Paralelismo entre pedidos, ordem dentro do pedido.

Pulsar vs Kafka (se perguntarem)

Pulsar separa computação (brokers) de armazenamento (BookKeeper), tem ACK individual por mensagem, NACK, DLQ e reentrega nativos, e multi-tenancy. Kafka trabalha com offset por partição e o paralelismo é limitado pelo número de partições.

Garantia de entrega

Pulsar entrega **at-least-once**. "Exactly-once ponta a ponta" não existe na prática: o que se faz é **at-least-once + consumidor idempotente** = efeito exatamente uma vez.

7 Webhooks

Os que enviamos (para o cliente final)

- Nascem na **mesma transação** do evento de negócio (tabela `webhook_deliveries`): é uma outbox de webhooks.
- Um **dispatcher** envia `POST` com JSON, reserva com `SKIP LOCKED`, timeout de 5s.
- 2xx = entregue. Qualquer outra coisa = retry com backoff até 6 tentativas, depois `failed` com reenvio manual.
- Cada tentativa fica em `webhook_attempts`: headers, status, resposta, erro, duração.

Assinatura (padrão Standard Webhooks)

```
webhook-id:      88cb082a-...      (igual em todos os retries)
webhook-timestamp: 1791299947
webhook-signature: v1,base64( HMAC-SHA256(segredo, id + "." + timestamp + "." + corpo) )
```

O cliente recalcula o HMAC sobre o **corpo cru** (byte a byte, sem re-serializar o JSON), compara com `timingSafeEqual` e recusa timestamps com mais de 5 minutos (evita **replay attack**).

Os que recebemos (da transportadora)

1. Validar a assinatura com o segredo combinado. Sem ela → **401**.
2. Gravar em `inbound_webhooks` com id único. Duplicata → **200** sem reprocessar.
3. Na mesma transação: pedido DELIVERED + webhook `order.delivered` para o cliente.
4. Responder rápido. Trabalho pesado vai para fila, não para dentro da requisição.

Riscos para mencionar

SSRF (cliente cadastrar URL interna), guardar o segredo criptografado, rotação de segredo (o header aceita várias assinaturas separadas por espaço), não seguir redirects.

8 Pix: SPI e mensagens ISO 20022

Peças do arranjo

Sigla	O que é
SPI	Sistema de Pagamentos Instantâneos, operado pelo Banco Central. Liquida cada Pix individualmente (liquidação bruta em tempo real), 24x7, nas contas PI que os participantes mantêm no BC.
DICT	Diretório de Identificadores de Contas Transacionais: traduz a chave Pix (CPF, e-mail, celular, aleatória) nos dados da conta.
PSP	Prestador de Serviço de Pagamento: o banco/instituição do pagador e o do recebedor.
RSFN	Rede do Sistema Financeiro Nacional, por onde trafegam as mensagens (XML no padrão ISO 20022).
EndToEndId	Identificador único da transação, com 32 caracteres: <code>E</code> + ISPB do pagador (8) + data e hora <code>AAAAMMDDHHmm</code> (12) + 11 caracteres. É a chave de idempotência natural do Pix.

As três mensagens

Mensagem	Nome ISO 20022	Para que serve no Pix
----------	----------------	-----------------------

pacs.008	FI to FI Customer Credit Transfer	Ordem de pagamento. PSP do pagador → SPI → PSP do recebedor. Leva valor, pagador, recebedor e o EndToEndId.
pacs.002	FI to FI Payment Status Report	Status. O recebedor responde se aceita (ACSP) ou rejeita (RJCT + código de motivo). Depois de liquidar, o SPI confirma a ambos (ACSC).
pacs.004	Payment Return	Devolução. Devolve total ou parcialmente um Pix já liquidado (erro, pedido do cliente, fraude/MED). Referencia o EndToEndId original e tem identificador próprio de devolução.

Fluxo de um Pix

Pagador → PSP pagador: valida saldo/limites, consulta DICT (chave → conta)
 PSP pagador —pacs.008→ SPI —pacs.008→ PSP recebedor
 PSP recebedor: valida conta —pacs.002 (ACSP ou RJCT)→ SPI
 SPI: liquida (debita conta PI do pagador, credita a do recebedor)
 SPI —pacs.002 (ACSC: liquidado)→ PSP pagador e PSP recebedor
 PSP recebedor credita o cliente · PSP pagador confirma ao pagador
 (se precisar devolver) PSP recebedor —pacs.004→ SPI → PSP pagador

Como os padrões do projeto se aplicam ao Pix

No Pix	Padrão
Debitar o cliente e enviar a pacs.008	Outbox: débito/reserva + mensagem a enviar na mesma transação. Nunca debitar sem a ordem sair, nunca mandar a ordem sem o débito.
Timeout esperando a pacs.002	Não gerar um novo EndToEndId e mandar de novo (seria um segundo Pix). Reenviar com o mesmo id ou consultar o status: idempotência pelo EndToEndId.
pacs.002 chegando duas vezes	Inbox: registrar EndToEndId + status processado; a segunda vez não credita de novo.
Crédito ao recebedor	Efeito + marcação de processado na mesma transação (como o consumidor do projeto).
Mostrar "pago" ao cliente	Só depois da pacs.002 ACSC (liquidado). Antes disso, "em processamento", exatamente como o paymentStatus do projeto: PENDING no POST, PAID na consulta e no webhook.
Avisar o lojista que o Pix caiu	Webhook assinado com retry (a API Pix do BC prevê webhooks para cobranças recebidas).
Devolução (pacs.004)	Novo evento ligado ao EndToEndId original; o total devolvido nunca pode passar do valor original (cheçar na mesma transação).
Ordem das mensagens de uma conta	Chave de ordenação (como o Key_Shared com aggregate_id).

Para não errar na entrevista

Pix é **irrevogável** depois de liquidado: não existe "estornar" a pacs.008, existe **devolução** (pacs.004), que é uma nova transferência no sentido contrário. Se não tiver certeza de um detalhe do regulamento (prazos exatos, códigos de motivo), diga que consultaria o Manual de Interfaces do SPI e o Regulamento Pix do Banco Central.

9 Perguntas prováveis (respostas curtas)

Por que não publicar direto no broker depois do commit?

Porque o processo pode cair entre o commit e o publish: evento perdido. A outbox deixa o evento durável junto com o dado.

Por que o POST não devolve PAID?

Porque a confirmação acontece depois, no consumidor. O POST devolve `PENDING`; o `PAID` chega pelo webhook `order.confirmed` ou pelo `GET /orders/:id`. Se precisasse de resposta "quase síncrona", daria para esperar alguns segundos (`Prefer: wait=5`) e cair para 202 + `PENDING` no timeout.

Isso garante exactly-once?

Não. Garante at-least-once. O "exatamente uma vez" vem do consumidor idempotente (inbox), que descarta duplicatas pelo eventId.

E se dois relays rodarem ao mesmo tempo?

`FOR UPDATE SKIP LOCKED`: cada um pega linhas diferentes. Há teste provando que 40 eventos com dois relays concorrentes são publicados 40 vezes, sem repetição.

E se o relay morrer depois de publicar e antes de marcar?

A linha fica `processing` até o lease expirar; outro ciclo publica de novo; o consumidor recebe a duplicata e descarta.

Como manter a ordem dos eventos?

Mesma chave por agregado (id do pedido) + assinatura `Key_Shared`: ordem por pedido, paralelismo entre pedidos.

O que acontece se o consumidor falhar?

A transação faz rollback (inclusive o registro em `processed_events`), o handler dá `NACK` e o Pulsar reentrega. Depois de 10 vezes, DLQ.

Como chamar uma API externa com segurança se ela não participa da transação?

Idempotency-Key estável (o eventId) + retry via `NACK`. A API do outro lado devolve o mesmo resultado para a mesma chave.

Como o cliente confia no webhook?

HMAC-SHA256 com segredo compartilhado sobre id + timestamp + corpo cru, comparação em tempo constante e janela de 5 minutos.

Como você testou?

Testes e2e contra Postgres real: atomicidade (rollback), broker fora do ar com backoff e dead letter, duplicata ignorada, queda após publicar, relays concorrentes, fluxo completo com transportadora e webhooks, assinatura inválida (401).

O que você melhoraria para produção?

Métricas e alertas (idade do evento mais antigo na outbox, tamanho da DLQ), tracing distribuído (OpenTelemetry), criptografia do segredo de webhook, proteção contra SSRF, CDC se a latência importar.

10 Glossário e checklist da demo

Termo	Em uma linha
Dual write	Gravar em dois sistemas sem transação comum

Roteiro da demo (painel em `localhost:3000`)

- `docker compose up -d`, `pnpm run db:migrate`,
`pnpm run start:dev`.
- Simular fluxo completo** e narrar os 10 passos.

Inbox	Tabela de eventos já processados pelo consumidor
Lease	"Reserva com prazo" para recuperar trabalho abandonado
Backoff + jitter	Esperas crescentes com aleatoriedade
DLQ	Fila de mensagens que esgotaram as tentativas
ACK / NACK	Confirma / devolve a mensagem ao broker
Ledger / entry	Arquivo de log do BookKeeper / registro nele
Cursor	Até onde a assinatura já confirmou
HMAC	Assinatura com segredo compartilhado
ISO 20022	Padrão internacional de mensagens financeiras

3. "Enviar 2x com a mesma chave": um pedido, duplicata 409.
4. Ligar "Broker fora do ar", criar pedidos, mostrar backoff; desligar e ver drenar.
5. Ligar "API da transportadora fora do ar": passo 7 vermelho, NACK e retry; desligar.
6. Aba **Pulsar**: producer, ledger, Key_Shared, cursor.
7. Aba **Webhooks**: assinatura, tentativas, "✓ assinatura válida".
8. "Testar atômidade": rollback desfaz pedido e evento.

Se algo falhar ao vivo: `docker ps` (containers no ar?) e `pnpm run db:migrate` (tabelas criadas?).